

Lab Test 2

[25 marks]**Coding time: 1.5 hours****General Instructions:**

1. You will perform the lab test on your personal laptop. You must disable any IDE extensions that suggest code, e.g., GitHub Copilot or IntelliCode, before the test. You may not use any local LLMs.
2. You may not communicate with anyone or access any network during the test. Remove earbuds, headsets, and smart watches.
3. You may refer to any file on your laptop during the test.
4. Make sure your code produces **exactly** the output shown in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspelling, etc., in the output.
5. Do not hardcode. We will use different test cases to test and grade your solutions.
6. Follow standard Python coding conventions (e.g., naming functions and variables).
7. A Python script file that cannot be executed will NOT be marked, and hence you will be awarded zero marks. You may wish to comment out the parts of your code that cause execution errors.
8. Include your name as the author in the comments of all your submitted source files. For example, include the following block of comments at the beginning of each source file you submit.

```
# Name: Lum Ting Wong  
# Email ID: lum.ting.wong.2025
```

Instructions on How to Submit Your Solutions:

1. Before the test begins, you will be instructed to start the Examenax exam and download the encrypted resource file from the exam appendix.
2. Wait for the password to decompress the ZIP file. Do not open the test paper until you receive the password.
3. After decompressing the file, **rename the folder** to your email ID. For example, if your SMU email is lum.ting.wong.2025@computing.smu.edu.sg, you should rename the folder to **LT2_lum.ting.wong.2025**

Save your solutions to this folder for submission later. You may be penalized for not following these instructions.

4. At the end of the test, await instructions to upload your solution to Examenax. Use the **zip_your_files.py** script included in the resource to package your solutions as a single ZIP file.

Alternatively, you may manually ZIP your files. For example, if your folder is named **LT2_lum.ting.wong.2025**, upload **LT2_lum.ting.wong.2025.zip**

5. Do not submit the Examenax exam until instructed by your invigilator.
6. After submitting the exam, close your laptop and wait quietly until the invigilator dismisses you.

Question 1

[8 Marks]

Part A (Difficulty Level *)

Implement a function called `get_specific_tuples`. This function takes two parameters:

- `tup_list (type: list[tuple])`: A list of tuples, where each tuple contains two or more integers.
- `num (type: int)`: A positive integer.

The function should return a **new list** containing only those tuples from `tup_list` whose sum of the **first** and **last** elements is greater than or equal to `num`.

The function should return an **empty list** if no tuple satisfies this condition or if `tup_list` is an empty list. Your code **must not modify** `tup_list`.

Examples:

```
a_list = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

- `get_specific_tuples(a_list, 5)` returns `[(4, 5, 6), (7, 8, 9)]`
- `get_specific_tuples(a_list, 15)` returns `[(7, 8, 9)]`
- `get_specific_tuples(a_list, 20)` returns `[]`
- `get_specific_tuples([], 10)` returns `[]`

```
b_list = [(1, 2, 3), (4, 5, 6, 0), (7, 8, 9, 10, 11, 12), (10, 4)]
```

- `get_specific_tuples(b_list, 14)` returns `[(7, 8, 9, 10, 11, 12), (10, 4)]`
- `get_specific_tuples(b_list, 15)` returns `[(7, 8, 9, 10, 11, 12)]`
- `get_specific_tuples(b_list, 20)` returns `[]`

Use `q1a_test.py` to test your code. We will only grade your `q1a.py` using a **different test script**.

Part B (Difficulty Level *)

You are given the helper function `clean_sentence()`.

```
def clean_sentence(sentence):  
    for ch in "!?,:;":  
        sentence = sentence.replace(ch, " ")  
    return sentence
```

This helper function takes a string as input and returns a new string where every punctuation character ("!?,.:;") in the input string is replaced with a space (" ").

For example:

- `clean_sentence("Abc!DEF?ghi")` returns `"Abc DEF ghi"`.

Implement a function called `extract_words` that takes one parameter:

- `sentence (type: str)`: A **non-empty string** containing two or more words separated by one punctuation character (!, ?, ., ,, ;, :). The first or last character in the string will not be a punctuation mark.

The function should return a list of words (type: `list[str]`) from sentence.

Examples:

- `extract_words("Hello!world?Python,rocks")` returns `['Hello', 'world', 'Python', 'rocks']`
- `extract_words("apple;banana:cherry")` returns `['apple', 'banana', 'cherry']`
- `extract_words("Go.Go!Go")` returns `['Go', 'Go', 'Go']`

Use `q1b_test.py` to test your code. We will only grade your `q1b.py` using a **different test script**.

Question 2

[6 Marks]

You are a lonely astronaut in a secret outpost on Mars in 1998. You are desperate to read the latest Harry Potter novel. A friend on Earth sends you the text over a very slow interplanetary data link, which your computer saves to a text file.

Part A (Difficulty Level **)

Implement a function called `decode_potter` in `q2a.py`. The function takes one parameter:

- `filename` (type: `str`): The filename of the latest download from the data link.

The file stores one sentence per line. Each line has the following format:

```
line = <sentence_length><sentence><random_data>
```

- `sentence_length`: A 3-digit decimal integer indicating the length of the sentence field, padded with '0'. For example, the number 32 is encoded as '032'. Note that `int('032') = 32`.
- `sentence`: One sentence from the novel, which is `sentence_length` characters long.
- `random_data`: Zero or more random characters that are not part of the novel.

For example:

```
line:          '018Finished yer examsMYSvkSjjmVcDUZuXEYxE'
sentence_length: '018'
sentence:       'Finished yer exams'
random_data:    'MYSvkSjjmVcDUZuXEYxE'
```

Note that the '<' and '>' in the format description are only to illustrate that one data field is next to another in the input string, but do not separate fields in the actual input string.

This function processes each line in the file and returns a `list[str]` that contains the sentence field of each message in the order that they appear in the file.

Example:

```
018Finished yer examsMYSvkSjjmVcDUZuXEYxE
000bnFZVgqsMkrv
013"Report him!"uzyFrNeElcMCZwSCDPi
022Has anyone seen a toadxO
```

The file `sentences-1.txt` has four lines. Line 2 has an empty sentence field with length 0.

In this case, `decode_potter('sentences-1.txt')` should return:

```
['Finished yer exams', '', '"Report him!"', 'Has anyone seen a toad']
```

Use `q2a_test.py` to test your code. We will only grade your `q2a.py` using a **different test script**.

Part B (Difficulty Level **)

Your belligerent neighbors, Lrrr and Morbo, don't like Harry Potter. They occasionally corrupt the data stream with random characters or make idle threats to convince you to leave.

Fortunately, you can detect a corrupted sentence using a checksum: an extra number that lets the receiver check if the sentence was changed since it was transmitted.



Figure 1: Your neighbors, Lrrr, Ruler of Omicron Persei 8 (left) and Morbo, social media influencer (right).

In this question, we will use a simple scheme: The checksum is the number of vowels in the novel text, where vowels are "AaEeIiOoUu".

The sender computes the checksum on the original `text` and concatenates it with the `text` to compose the `sentence` field. The revised message format:

```
line = <sentence_length><sentence><random_data>
where
<sentence> = <text><transmitted_checksum>
```

The `transmitted_checksum` is a 2-digit decimal integer padded with '0'.

When you read a `line`:

1. Extract the `sentence` field.
2. Separate the `text` and the `transmitted_checksum`.
3. Compute the checksum for the `text` you received.
4. Compare the computed checksum to the `transmitted_checksum`.

Implement the function `decode_potter_checksum` in `q2b.py`. The function takes one parameter:

- `filename` (type: `str`): The filename of the latest download from the data link. The message format now includes a checksum for each sentence.

This function returns a list of tuples (type: `list[tuple(str, int, bool)]`), with one tuple for each line, in the order they appear in the file. Each tuple contains:

- The `text` part of the `sentence` field.
- The number of vowels ("AaEeIiOoUu") in the `text` field.
- A Boolean flag that is `True` if the actual checksum equals the `transmitted_checksum`, and `False` otherwise.

Note: Lrrr and Morbo may corrupt only the `text` part of the message.

Hint: You may import the `decode_potter` function implemented in Q2 Part A to extract the `sentence` field for each `line` in a file. Split each sentence into `text` and `transmitted_checksum` components, count the vowels in `text`, and compare the checksums.

Example:

```
032Never mess with Morbo!!, Harry07EdppsfeRXWVYtDuhG
038Professor Lrrrrrrrrrr sniffed angrily10z
043Harry saw the Ape-Man looked very worried12dKRQYZIMzW
024Has anyone seen Morbo?091NIzPslKKelpZAXBZdT
065The toadless boy was back, but this time he had a girl with him17owIuAILIOUOCeVltQSbK
```

The file sentences-4.txt has five lines.

In this case, `decode_potter_checksum('sentences-4.txt')` should return:

```
[('Never mess with Morbo!!, Harry', 7, True),
 ('Professor Lrrrrrrrrrr sniffed angrily', 7, False),
 ('Harry saw the Ape-Man looked very worried', 13, False),
 ('Has anyone seen Morbo?', 8, False),
 ('The toadless boy was back, but this time he had a girl with him', 17, True)]
```

- Lines 2, 3, and 4 have non-matching checksums and are marked `False`.
- Line 1 is clearly corrupted, but the number of vowels in the corrupted text did not change, so the message is therefore marked `True`.
- Line 5 is an actual sentence from the novel and is marked `True`.

Use `q2b_test.py` to test your code. We will only grade your `q2b.py` using a **different test script**.

Question 3**[5 Marks]****Part A (Difficulty Level **)**

Implement a function called `read_weather_data` in `q3a.py`. The function takes one parameter:

- `filename` (type: `str`): The name of the input file containing daily weather logs for various cities.

The input text file consists of one weather record per line. Each line uses a pipe character (`'|'`) to separate the five fields:

1. Date in `'YYYY-MM-DD'` format.
2. City name
3. Maximum temperature in Celsius, as a decimal number.
4. Minimum temperature in Celsius, as a decimal number.
5. Precipitation in millimeters, as a decimal number.

An example input text file named `'weather_logs.txt'` contains the following lines in no particular order.

```
2023-10-01|Rivertown|25.5|15.1|0.0
2023-11-15|Sunnyvale|19.9|8.5|10.2
2023-10-01|Sunnyvale|28.1|16.3|0.0
2023-10-02|Rivertown|24.9|14.0|5.5
2023-11-16|Sunnyvale|18.5|7.9|12.1
2023-10-02|Sunnyvale|27.5|15.8|2.1
```

Your function must return a dictionary with:

- `key`: The city name (e.g., `'Rivertown'`, `'Sunnyvale'`)
- `value`: A list of tuples containing the weather information for a specific day in the same order as their corresponding data appears in the input text file.

Each tuple has the form:

```
((year, month, day), max_temp, min_temp, precipitation)
```

Example:

Given the file 'weather_logs.txt', `read_weather_data('weather_logs.txt')` returns:

```
{'Rivertown': [
    ((2023, 10, 1), 25.5, 15.1, 0.0), ((2023, 10, 2), 24.9, 14.0, 5.5)],
'Sunnyvale': [
    ((2023, 11, 15), 19.9, 8.5, 10.2), ((2023, 10, 1), 28.1, 16.3, 0.0),
    ((2023, 11, 16), 18.5, 7.9, 12.1), ((2023, 10, 2), 27.5, 15.8, 2.1)]}
```

Use `q3a_test.py` to test your code. Your `q3a.py` should contain the function definition of `read_weather_data`. We will only grade your `q3a.py` using a **different test script**.

Part B (Difficulty Level **/***)

Implement a function called `analyze_weather_patterns` in `q3b.py` to calculate the average maximum temperature and total precipitation for cities over a specific period.

The function takes three parameters:

- `filename` (type: `str`): The name of a file containing weather logs as described in Q3 Part A.
- `start_date` (type: `tuple(int)`): The analysis starting date, (year, month, day), inclusive.
- `end_date` (type: `tuple(int)`): The analysis ending date, (year, month, day), inclusive.

The `start_date` and the `end_date` may be *any* valid date from January 1, 1970, to the present.

Your function must return a dictionary with statistics only for cities with data within the date range, where:

- `key`: The city name (e.g., 'Rivertown', 'Sunnyvale')
- `value`: A tuple containing three statistics for that city:
(`record_count`, `avg_max_temperature`, `total_precipitation`)

Note: The average maximum temperature and total precipitation should be rounded to one decimal place. You may want to use the built-in `round` function for this.

You may import and use the function implemented in Q3 Part A.

Examples:

If the function is invoked like this:

```
analyze_weather_patterns('weather_logs.txt', (2023, 11, 15), (2023, 11, 16))
```

It should return:

```
{'Sunnyvale': (2, 19.2, 22.3)}
```

Note: For Sunnyvale, the average max temperature is $(19.9 + 18.5) / 2 = 19.2$. The total precipitation is $10.2 + 12.1 = 22.3$.

If the function is invoked like this:

```
analyze_weather_patterns('weather_logs.txt', (2023, 10, 1), (2023, 10, 31))
```

It should return:

```
{'Rivertown': (2, 25.2, 5.5), 'Sunnyvale': (2, 27.8, 2.1)}
```

Note: For Rivertown, the average max temperature is $(25.5 + 24.9) / 2 = 25.2$. For Sunnyvale, it is $(28.1 + 27.5) / 2 = 27.8$.

If the function is invoked like this:

```
analyze_weather_patterns('weather_logs.txt', (2023, 10, 3), (2023, 11, 15))
```

It should return:

```
{'Sunnyvale': (1, 19.9, 10.2)}
```

Use `q3b_test.py` to test your code. Your `q3b.py` should contain the function definition of `analyze_weather_patterns`. We will only grade your `q3b.py` using a **different test script**.

Question 4

[6 Marks]

A frog must navigate a map that represents a pond, starting at point A and ending at point B. The map may be one-dimensional or two-dimensional.

- There are several snake nests located along the paths between A and B, excluding the endpoints.
- If the frog steps into a snake nest, it will be eaten by the snake.
- The frog can jump 1, 2, or 3 steps at a time to the right in a 1D map.
- The frog can jump 1, 2, or 3 steps at a time to the right **or** 1, 2, or 3 steps at a time down in a 2D map.
- The frog cannot jump to the left or up.

The problem is to determine the **possible safe routes and corresponding jump sequences** for the frog to reach point B.

Part A (Difficulty Level ***)

In `q4a.py`, implement a function called `help_frog_against_snakes` which takes one parameter:

- `map (type: str)`: A string that stores the one-dimensional map from point A to point B.

The `map` string starts with the character "A" and ends with the character "B". The substring between "A" and "B", with a minimum length of 2, consists of "0" (zero) and/or "*", where "*" represents a snake nest and "0" represents a safe position.

The following are example strings and their corresponding one-dimensional maps:

Input map string	Actual Map
A00**0*00B	A 0 0 * * 0 * 0 0 B
A0000B	A 0 0 0 0 B
A000*****B	A 0 0 0 * * * * * B

The function returns a tuple of (total, jump_sequences),

- **total (type: int):** The total number of jump sequences where the frog survives the trip from point A to point B. This value will be zero if there is no safe jump sequence.
- **jump_sequences (type: list[list[int]]):** A list of all the possible jump sequences where the frog survives the trip from point A to point B. If there are no safe jump sequences, set it to the empty list. Each element of this list is a list containing the number of steps (i.e., 1, 2, or 3) for each move in a jump sequence.
- The order of the lists in jump_sequences does not matter.

The following are examples of input maps and corresponding outputs:

Input map string	total	jump_sequences
A00**0*00B	6	[[2, 3, 2, 2], [1, 1, 3, 2, 2], [2, 3, 3, 1], [1, 1, 3, 3, 1], [2, 3, 2, 1, 1], [1, 1, 3, 2, 1, 1]]
A0000B	13	[[2, 3], [1, 1, 3], [3, 2], [1, 2, 2], [2, 1, 2], [1, 1, 1, 2], [1, 3, 1], [2, 2, 1], [1, 1, 2, 1], [3, 1, 1], [1, 2, 1, 1], [2, 1, 1, 1], [1, 1, 1, 1, 1]]
A000*****B	0	[]

Use `q4a_test.py` to test your code. Your `q4a.py` should contain the function definition of `help_frog_against_snakes`. We will only grade your `q4a.py` using a **different test script**.

Part B (Difficulty Level ***)

In `q4b.py`, implement a function called `help_frog_against_snakes_2` which takes one parameter:

- **map (type: list[str]):** A list of strings that represent a two-dimensional map, where each string is one row of the map. There are at least three strings in the list, and all have the same length of at least three.

The first string starts with the character "A" and the last string ends with the character "B". The "*" character represents a snake nest, and "0" (zero) represents a safe position.

The following are example map lists and their corresponding two-dimensional maps:

Input map	Actual Map																
['A00*', '00*0', '**0B']	<table><tr><td>A</td><td>0</td><td>0</td><td>*</td></tr><tr><td>0</td><td>0</td><td>*</td><td>0</td></tr><tr><td>*</td><td>*</td><td>0</td><td>B</td></tr></table>	A	0	0	*	0	0	*	0	*	*	0	B				
A	0	0	*														
0	0	*	0														
*	*	0	B														
['A***', '*000', '*000', '*00B']	<table><tr><td>A</td><td>*</td><td>*</td><td>*</td></tr><tr><td>*</td><td>0</td><td>0</td><td>0</td></tr><tr><td>*</td><td>0</td><td>0</td><td>0</td></tr><tr><td>*</td><td>0</td><td>0</td><td>B</td></tr></table>	A	*	*	*	*	0	0	0	*	0	0	0	*	0	0	B
A	*	*	*														
*	0	0	0														
*	0	0	0														
*	0	0	B														
['A00', '000', '00B']	<table><tr><td>A</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>B</td></tr></table>	A	0	0	0	0	0	0	0	B							
A	0	0															
0	0	0															
0	0	B															

The function returns a **tuple** of (total, jump_sequences),

- **total** (type: int): The total number of jump sequences where the frog survives the trip from point A to point B. This value will be zero if there is no safe jump sequence.
- **jump_sequences** (type: list[list[tuple(int, str)]]): A list of all possible jump sequences where the frog survives the trip from point A to point B. If there are no safe jump sequences, set it to the empty list.
Each element of this list is a list of tuples (steps, direction) where steps can be 1, 2, or 3, and direction can be 'd' (down) or 'r' (right).
- The order of the lists in jump_sequences does not matter.

The following are examples of input maps and corresponding outputs:

Input map list	total	jump_sequences
['A00*', '00*0', '**0B']	5	[[(1, 'd'), (3, 'r'), (1, 'd')], [(1, 'r'), (1, 'd'), (2, 'r'), (1, 'd')], [(1, 'd'), (1, 'r'), (2, 'r'), (1, 'd')], [(2, 'r'), (2, 'd'), (1, 'r')], [(1, 'r'), (1, 'r'), (2, 'd'), (1, 'r')]]
['A***', '*000', '*000', '*00B']	0	[]

Lab Test 2 – COR-IS1704

['A00', '000', '00B']	14	[[(2, 'r'), (2, 'd')], [(1, 'r'), (1, 'r'), (2, 'd')], [(2, 'r'), (1, 'd'), (1, 'd')], [(1, 'r'), (1, 'r'), (1, 'd'), (1, 'd')], [(1, 'd'), (2, 'r'), (1, 'd')], [(1, 'r'), (1, 'd'), (1, 'r'), (1, 'd')], [(1, 'd'), (1, 'r'), (1, 'r'), (1, 'd')], [(2, 'd'), (2, 'r')], [(1, 'd'), (1, 'd'), (2, 'r')], [(1, 'r'), (2, 'd'), (1, 'r')], [(1, 'r'), (1, 'd'), (1, 'd'), (1, 'r')], [(1, 'd'), (1, 'r'), (1, 'd'), (1, 'r')], [(2, 'd'), (1, 'r'), (1, 'r')], [(1, 'd'), (1, 'd'), (1, 'r'), (1, 'r')]]
-----------------------------	----	---

Use `q4b_test.py` to test your code. Your `q4b.py` should contain the function definition of `help_frog_against_snakes_2`. We will only grade your `q4b.py` using a **different test script**.

END OF PAPER